
Module 02: HTML & CSS for Svelte

Master the building blocks of the web and learn how Svelte supercharges your styling workflow.

Revolution Trading Pros - Web Dev Course

Overview

HTML and CSS are the foundation of every web application. In this module, you will learn semantic HTML5 elements, modern CSS techniques including Flexbox and Grid, and how Svelte scopes styles to individual components. By the end, you will build a responsive card layout from scratch.

Even if you have some HTML and CSS experience, this module covers Svelte-specific patterns that are essential for the rest of the course.

Semantic HTML5

Semantic HTML uses elements that clearly describe their meaning to both the browser and the developer. Using semantic elements improves accessibility, SEO, and code readability.

Common Semantic Elements

- ❏ `<header>`: Introductory content or navigation links. Typically contains the site logo and main navigation.
- ❏ `<nav>`: A section of navigation links. Use for primary site navigation, not every group of links.
- ❏ `<main>`: The dominant content of the page. There should be only one `<main>` per page.
- ❏ `<article>`: Self-contained content that could be distributed independently (blog post, news article, product card).
- ❏ `<section>`: A thematic grouping of content, typically with a heading.
- ❏ `<aside>`: Content tangentially related to the surrounding content (sidebars, call-out boxes).
- ❏ `<footer>`: Footer content, typically containing copyright info, links, and contact details.

HTML

```
<header>
  <nav>
    <a href="/">Home</a>
    <a href="/about">About</a>
    <a href="/contact">Contact</a>
  </nav>
</header>

<main>
  <article>
    <h1>My First Article</h1>
    <p>This is the article content.</p>
  </article>
</main>

<footer>
  <p>&copy; 2025 My Website</p>
</footer>
```

Avoid using `<div>` for everything. Ask yourself: 'Does this element have a specific meaning?' If yes, use the appropriate semantic element.

CSS Scoped Styles in Svelte

One of Svelte's most powerful features is automatic CSS scoping. When you write styles in a Svelte component's `<style>` block, those styles only apply to that component. No more CSS class name collisions!

Svelte

```
<!-- Button.svelte -->
<button>Click me</button>

<style>
  /* This ONLY affects buttons in this component */
  button {
    background: #ff3e00;
    color: white;
    border: none;
    padding: 0.5rem 1rem;
    border-radius: 4px;
    cursor: pointer;
  }

  button:hover {
    background: #cc3200;
  }
</style>
```

Under the hood, Svelte adds unique class attributes to your elements and modifies your selectors to include these classes. This means you can write simple, readable CSS without worrying about global conflicts.

When You Need Global Styles

Sometimes you do need global styles (for body, reset styles, or third-party library overrides). Use the `:global()` modifier:

```
<style>
  /* Scoped - only this component */
  p { color: blue; }

  /* Global - affects everything */
  :global(body) {
    margin: 0;
    font-family: system-ui, sans-serif;
  }

  /* Global within a scoped parent */
  div :global(strong) {
    color: red;
  }
</style>
```

Flexbox Layout

Flexbox is a one-dimensional layout method for arranging items in rows or columns. It excels at distributing space and aligning items within a container.

Core Concepts

- Flex Container: The parent element with `display: flex`. Controls the overall layout direction and alignment.
- Flex Items: The direct children of a flex container. Can be individually sized and ordered.
- Main Axis: The primary axis along which flex items are laid out (horizontal for row, vertical for column).
- Cross Axis: Perpendicular to the main axis.

CSS

```
.container {
  display: flex;
  justify-content: space-between; /* Main axis alignment */
  align-items: center;           /* Cross axis alignment */
  gap: 1rem;                    /* Space between items */
  flex-wrap: wrap;              /* Allow wrapping */
}

.item {
  flex: 1;                      /* Grow equally */
  min-width: 200px;             /* Minimum size before wrap */
}
```

Common Flexbox Patterns

Navigation bar with logo left and links right:

CSS

```
nav {
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: 1rem 2rem;
}
```

Centering an element both horizontally and vertically:

CSS

```
.center-me {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  min-height: 100vh;  
}
```

CSS Grid Layout

CSS Grid is a two-dimensional layout system. While Flexbox handles one dimension (row or column), Grid handles both simultaneously, making it perfect for page layouts and complex component designs.

Defining a Grid

CSS

```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
  grid-template-rows: auto;             /* Rows size to content */
  gap: 1.5rem;                          /* Row and column gap */
}

/* Responsive grid */
.responsive-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 1.5rem;
}
```

The `repeat(auto-fit, minmax(280px, 1fr))` pattern is incredibly powerful. It creates as many columns as will fit, each at least 280px wide, and distributes remaining space equally. This creates a fully responsive grid without any media queries.

Grid Areas

For complex layouts, you can name grid areas:

CSS

```
.layout {  
  display: grid;  
  grid-template-areas:  
    "header header"  
    "sidebar main"  
    "footer footer";  
  grid-template-columns: 250px 1fr;  
  grid-template-rows: auto 1fr auto;  
  min-height: 100vh;  
}  
  
.header { grid-area: header; }  
.sidebar { grid-area: sidebar; }  
.main    { grid-area: main; }  
.footer  { grid-area: footer; }
```

CSS Custom Properties (Variables)

CSS Custom Properties (also called CSS variables) allow you to define reusable values that can be updated dynamically. They are essential for theming and maintaining consistent design systems.

CSS

```
:root {
  --color-primary: #ff3e00;
  --color-secondary: #7c3aed;
  --color-bg: #ffffff;
  --color-text: #333333;
  --font-body: system-ui, -apple-system, sans-serif;
  --font-mono: "Fira Code", monospace;
  --spacing-sm: 0.5rem;
  --spacing-md: 1rem;
  --spacing-lg: 2rem;
  --radius: 8px;
  --shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}

/* Usage */
.card {
  background: var(--color-bg);
  color: var(--color-text);
  padding: var(--spacing-lg);
  border-radius: var(--radius);
  box-shadow: var(--shadow);
  font-family: var(--font-body);
}
```

Custom properties cascade like any other CSS property, so you can override them at any level of specificity. This makes dark mode implementation straightforward:

CSS

```
[data-theme="dark"] {
  --color-bg: #1a1a2e;
  --color-text: #e0e0e0;
  --shadow: 0 2px 8px rgba(0, 0, 0, 0.4);
}
```

Responsive Design

Responsive design ensures your application looks great on all screen sizes. The key principles are fluid layouts, flexible images, and media queries.

The Viewport Meta Tag

This must be in your HTML head (SvelteKit includes it by default in app.html):

HTML

```
<meta name="viewport" content="width=device-width, initial-scale=1">
```

Mobile-First Media Queries

Write base styles for mobile, then add complexity for larger screens:

CSS

```
/* Base styles (mobile) */
.container {
  padding: 1rem;
}

.grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 1rem;
}

/* Tablet (768px+) */
@media (min-width: 768px) {
  .container { padding: 2rem; }
  .grid { grid-template-columns: repeat(2, 1fr); }
}

/* Desktop (1024px+) */
@media (min-width: 1024px) {
  .container { max-width: 1200px; margin: 0 auto; }
  .grid { grid-template-columns: repeat(3, 1fr); }
}
```

Responsive Units

rem: Relative to root font size. Use for font sizes, spacing, and widths.

em: Relative to parent font size. Use for padding/margin relative to the current element's text.

vw/vh: Viewport width/height. Use for hero sections and full-screen elements.

clamp(): A modern function for fluid typography: `font-size: clamp(1rem, 2.5vw, 2rem);`

%: Percentage of parent. Use for flexible containers within a defined width.

Module 2 Exercise

Exercise: Responsive Card Layout

Put what you have learned into practice with this hands-on exercise.

- %I Step 1: Create a new Svelte component called `CardGrid.svelte` in `src/lib/components/`.
- %I Step 2: Define CSS custom properties for colors, spacing, and border-radius in a `:root` block.
- %I Step 3: Create a card component with an image placeholder, title, description, and a button.
- %I Step 4: Use CSS Grid with `repeat(auto-fit, minmax(280px, 1fr))` to make the layout responsive.
- %I Step 5: Style the cards with hover effects (transform, shadow transitions).
- %I Step 6: Add a navigation bar above the grid using Flexbox.
- %I Step 7: Test on different screen sizes using the browser's responsive design mode (Cmd+Shift+M in Chrome).
- %I Step 8: Commit your changes to Git with a descriptive message.

Summary

- %I Semantic HTML improves accessibility, SEO, and code readability.
- %I Svelte scopes CSS automatically - no class name collisions.
- %I Flexbox is ideal for one-dimensional layouts (navbars, centering).
- %I CSS Grid handles two-dimensional layouts (page layouts, card grids).
- %I CSS Custom Properties enable theming and consistent design systems.
- %I Mobile-first responsive design ensures great experiences on all devices.